

Resumen de Arquitectura de Computadores

Clase del 7 de mayo de 2026 - AVX, SIMD, distancia euclídea y arranque del trabajo

Idea central: La clase continúa el tema de punto flotante empaquetado con AVX/SIMD. Primero se revisan instrucciones vectoriales, movimiento alineado y comparaciones empaquetadas. Luego se implementa y depura un ejemplo de distancia euclídea entre dos puntos usando registros YMM. Al final se introduce el trabajo: programar con funciones y stack frame el algoritmo extendido de Euclides para inversos multiplicativos modulares.

1. Continuación de AVX: aritmética empaquetada

El enfoque de la clase es dejar de calcular un dato a la vez y pasar a calcular varios datos en paralelo dentro de un mismo registro. En lugar de usar una instrucción para un solo número flotante, AVX permite empaquetar varios flotantes en registros XMM/YMM/ZMM y aplicar una operación a todos sus campos o lanes.

Registros a recordar: XMM trabaja con 128 bits, YMM con 256 bits y ZMM con 512 bits. Si se usan datos double de 64 bits, un YMM puede contener 4 doubles y un ZMM puede contener 8 doubles. Si se usan floats de 32 bits, caben más elementos.

2. Suma horizontal: `vhaddps` / `vhaddpd`

Se retomó la instrucción de suma horizontal. La idea es que, en vez de sumar posición por posición entre dos registros, se suman pares internos dentro de los registros. En la diapositiva se mostró un ejemplo con:

```
vhaddps xmm3, xmm1, xmm2
```

Ejemplo conceptual con single precision:

- `xmm1` = [0.1, 0.2, 0.3, 0.4]
- `xmm2` = [0.5, 0.6, 0.7, 0.8]
- resultado = [0.1+0.2, 0.5+0.6, 0.3+0.4, 0.7+0.8] = [0.3, 1.1, 0.7, 1.5]

Ojo: En YMM, muchas instrucciones horizontales trabajan por mitades internas de 128 bits. Esto puede producir resultados intermedios repetidos o ubicados en lanes específicos. En el ejemplo de distancia, el valor correcto aparece repetido en las últimas posiciones después de combinar los resultados.

3. Instrucciones FP empaquetadas comunes

La clase mostró una tabla de instrucciones de punto flotante empaquetado. El patrón general es que la letra `s` suele referirse a single precision y `d` a double precision.

Instrucción	Uso principal
<code>vaddp[s d]</code>	Suma empaquetada
<code>vsubp[s d]</code>	Resta empaquetada
<code>vmulp[s d]</code>	Multiplicación empaquetada
<code>vdivp[s d]</code>	División empaquetada

vandp[s d]	AND bitwise sobre campos empaquetados
vsqrtp[s d]	Raíz cuadrada empaquetada
vminp[s d]	Mínimo por lane
vmaxp[s d]	Máximo por lane
vmovap[s d]	Movimiento alineado
vorp[s d]	OR bitwise
vldmxcsr	Carga del registro MXCSR

4. Movimiento alineado: vmovap[s|d]

La instrucción vmovap[s|d] permite mover vectores de números de punto flotante desde memoria hacia registros AVX o desde registros hacia memoria, pero exige que el bloque de datos esté alineado.

- Cada byte tiene una dirección en RAM.
- La memoria se accede en grupos de bytes para mejorar el tiempo de acceso.
- Si n es el tamaño de palabra, una dirección está alineada cuando es divisible entre n.
- Un grupo de datos está alineado cuando su primer byte inicia en un vecindario de palabra.
- Si se usa movimiento alineado con datos mal alineados, el programa puede fallar o comportarse de forma incorrecta al ejecutar la instrucción.

Regla práctica: Para el trabajo, si se declaran arreglos o vectores que se van a mover con vmovapd/vmovaps, hay que cuidar que el tamaño declarado, el tipo de datos y el registro usado coincidan. No conviene mezclar qword, ymm/zmm y cantidad de elementos sin revisar cuántos bytes se están leyendo.

5. Comparaciones empaquetadas flotantes

También se revisaron instrucciones de comparación empaquetada. La forma general vista en clase fue:

```
vcmp[ps|pd] pac_regd, pac_reg1, pac_reg2, COMP
```

La comparación se hace lane por lane. El resultado no es 1 o 0 como entero normal, sino una máscara de bits:

- Si la comparación es falsa, el lane de resultado queda en 0x00000000 para single o 0x0000000000000000 para double.
- Si la comparación es verdadera, el lane queda lleno de unos: 0xFFFFFFFF para single o 0xFFFFFFFFFFFFFFFF para double.
- COMP indica el tipo de comparación; en los ejemplos se usó 0 para igualdad y 1 para menor que.

Ejemplo mostrado con single precision:

```
vcmpsps xmm2, xmm0, xmm1, 0 ; packed SPFP compare for EQ
```

```
xmm0 = [ 4.125, 2.375, -72.5, 44.125 ]
xmm1 = [ 8.625, 2.375, -72.5, 15.875 ]
xmm2 = [ 0x00000000, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000 ]
```

Ejemplo mostrado con double precision:

```
vcmpdpd xmm2, xmm0, xmm1, 1 ; packed DPFP compare for LT
```

```
xmm0 = [ 4.125, 72.5 ]
xmm1 = [ 6.125, 2.375 ]
```

```
xmm2 = [ 0xFFFFFFFFFFFFFFFF, 0x0000000000000000 ]
```

6. Ejemplo principal: distancia euclídea en SIMD

El ejemplo práctico de la clase fue calcular la distancia euclídea entre dos puntos, pero usando instrucciones SIMD en lugar de hacer cada resta, multiplicación y suma por separado.

Fórmula: $|P1P2| = \sqrt{(x2 - x1)^2 + (y2 - y1)^2 + (z2 - z1)^2}$

Datos usados en el ejemplo:

```
.data
Punto1  qword 2.5, 3.5, 4.5, 0.0
Punto2  qword -11.34, 7.281, -0.06, 0.0
Distancia qword ?,?,?,?
```

Código central visto en Visual Studio:

```
.code
main PROC
    vmovapd    ymm0, Punto1
    vmovapd    ymm1, Punto2
    vsubpd     ymm0, ymm1, ymm0
    vmulpd     ymm0, ymm0, ymm0
    vhaddpd    ymm0, ymm0, ymm0
    vbroadcastsd ymm1, xmm0
    vaddpd     ymm0, ymm0, ymm1
    vsqrtpd    ymm0, ymm0
    vmovapd    Distancia, ymm0

    call ExitProcess
main ENDP
END
```

Lectura línea por línea:

Instrucción	Qué hace
vmovapd ymm0, Punto1	Carga los cuatro valores del primer punto en YMM0.
vmovapd ymm1, Punto2	Carga los cuatro valores del segundo punto en YMM1.
vsubpd ymm0, ymm1, ymm0	Resta lane por lane. Produce diferencias entre coordenadas. El signo no importa luego porque se eleva al cuadrado.
vmulpd ymm0, ymm0, ymm0	Eleva al cuadrado cada diferencia.
vhaddpd ymm0, ymm0, ymm0	Suma horizontalmente por pares. Aquí hay que recordar el comportamiento por lanes/mitades internas.
vbroadcastsd ymm1, xmm0	Toma un double escalar y lo replica en los lanes de YMM1.
vaddpd ymm0, ymm0, ymm1	Suma el subtotal replicado con los subtotales existentes.
vsqrtpd ymm0, ymm0	Calcula la raíz cuadrada de cada lane.
vmovapd Distancia, ymm0	Guarda los resultados en memoria.

Resultado aproximado: Para Punto1 = (2.5, 3.5, 4.5) y Punto2 = (-11.34, 7.281, -0.06), la distancia correcta es aproximadamente 15.0544. Durante la depuración se observan otros valores auxiliares en algunos lanes; lo importante es identificar cuál lane contiene el resultado correcto según la secuencia usada.

7. Depuración en Visual Studio

- Se ejecuta paso a paso para ver cómo cambian YMM0, YMM1 y la memoria.
- La ventana de registros muestra valores en hexadecimal, por lo que puede ser difícil leer directamente los doubles.
- La ventana de memoria permite interpretar los mismos bytes como números reales y verificar los resultados esperados.
- La clase comparó el cálculo con Excel para confirmar la distancia y revisar errores de signo o de agrupación.

Detalle de código: En las capturas del video aparece `includelib \Windows\System32\kernel132.dll`. Ese nombre tiene pinta de typo; lo correcto normalmente es `kernel32.dll`.

8. XMM, YMM y ZMM: cuidado al cambiar el tamaño

En una parte de la clase se prueba cambiar registros YMM por ZMM. La idea es recordar que al aumentar el tamaño del registro también aumenta la cantidad de datos leídos, operados y mostrados en el depurador.

- XMM = 128 bits: 2 doubles o 4 floats.
- YMM = 256 bits: 4 doubles o 8 floats.
- ZMM = 512 bits: 8 doubles o 16 floats.
- Si se cambia de YMM a ZMM, también se debe revisar la declaración de datos, el alineamiento y si realmente se tienen suficientes elementos válidos en memoria.

9. Paralelización de una suma

Hacia el final se introduce la idea de partir una sumatoria en bloques que quepan en registros vectoriales. La forma conceptual fue:

$$\sum_{j=1}^n x_j = \sum_{k=1}^{\lceil n/8 \rceil} X_k + Y$$

Interpretación práctica:

- x_j representa los elementos individuales.
- X_k representa paquetes o bloques de 8 elementos que se pueden procesar juntos.
- Y representa la parte restante cuando n no es múltiplo exacto de 8.
- La idea es reducir la cantidad de iteraciones usando operaciones empaquetadas.

10. Arranque del trabajo: algoritmo extendido de Euclides

Al final de la clase se introduce el trabajo siguiente. La consigna escrita en pantalla fue programar en ensamblador, usando funciones y stack frame, el algoritmo extendido de Euclides para encontrar inversos multiplicativos modulares.

Conceptos mínimos para entender el trabajo:

- La división entera se expresa como $a = q \cdot b + r$, donde q es el cociente y r es el residuo.
- Un inverso multiplicativo modular de k módulo p es un número x tal que $(k \cdot x) \bmod p = 1$.
- Ejemplo visto: si $p = 5$, entonces $(2 \cdot 3) \bmod 5 = 1$. Por eso, 3 es inverso de 2 módulo 5 y 2 es inverso de 3 módulo 5.
- El inverso existe cuando k y p son coprimos, es decir, cuando $\text{MCD}(k, p) = 1$.
- El algoritmo extendido de Euclides no solo calcula el MCD, sino también coeficientes que permiten obtener el inverso modular.

Para el trabajo: Lo que probablemente quiere el profesor no es solo que el resultado matemático funcione, sino que se usen módulos/procedimientos con stack frame, paso de parámetros, valor retornado y depuración clara en Visual Studio.

11. Preguntas rápidas tipo repaso

Pregunta	Respuesta corta
¿Qué significa empaquetado?	Que un registro contiene varios números y una instrucción opera sobre todos sus lanes.
¿Qué hace vhaddpd?	Realiza sumas horizontales entre pares de doubles dentro de registros empaquetados.
¿Qué devuelve una comparación empaquetada?	Una máscara por lane: todos los bits en 1 si la condición es verdadera, o todos en 0 si es falsa.
¿Por qué importa el alineamiento?	Porque instrucciones como vmovapd/vmovaps esperan que el bloque de memoria empiece en una dirección adecuada.
¿Qué representa Y en la sumatoria vectorizada?	El residuo o parte restante que no llena un paquete completo.
¿Qué es un inverso modular?	Un x tal que $(k \cdot x) \bmod p = 1$.
¿Qué relación tiene Euclides extendido con el inverso modular?	Permite obtener coeficientes; cuando el MCD es 1, uno de esos coeficientes sirve como inverso modular ajustado al módulo.

12. Mini chuleta de instrucciones de esta clase

Instrucción	Uso
vmovapd ymm0, Punto1	mueve doubles alineados desde memoria a YMM0
vsubpd ymm0, ymm1, ymm0	resta empaquetada de doubles
vmulpd ymm0, ymm0, ymm0	multiplica cada lane por sí mismo
vhaddpd ymm0, ymm0, ymm0	suma horizontal de doubles
vbroadcastsd ymm1, xmm0	replica un double escalar en un registro vectorial
vaddpd ymm0, ymm0, ymm1	suma empaquetada de doubles
vsqrtpd ymm0, ymm0	raíz cuadrada por lane
vcmpsps/vcmppd	comparación empaquetada de floats/doubles